



Propiedades de Algoritmo

Correctitud

proceso bien definido para resolver
 través de un conjunto *finito* de
 que transforman entradas en s
 ón *general* a un problema “bien
 [Cormen et. al., 2009]

- **Algoritmo:** proceso bien definido para resolver problemas, planteado a través de un conjunto *finito* de pasos *lógicamente secuenciados* que transforman entradas en salidas. Debe brindar solución *general* a un problema “bien especificado” [Cormen et. al., 2009]

haga lo que tiene que hacer, p
válidas debe generar una salida
haga buen uso de los recursos
o como de espacio

- ▶ **correcto:** que haga lo que tiene que hacer, para todo conjunto de entradas válidas debe generar una salida válida
- ▶ **eficiente:** que haga buen uso de los recursos, tanto de procesamiento como de espacio

Algoritmos

on construcciones que permiten
o de una condición (programa)
eba: escenarios específicos en lo

Varias formas:

- ▶ *assert*: que son construcciones que permiten verificar el cumplimiento de una condición (programa)
- ▶ *casos de prueba*: escenarios específicos en los cuales se ejecuta el programa

Algoritmos

al: definida por Charles Antony
nio Turing (1980)

are proporciona una serie de re
sobre la corrección de program
lógica matemática.

- Logica de Hoare proporciona una serie de reglas de inferencia para razonar sobre la corrección de programas imperativos con el rigor de la lógica matemática.

goritmos

característica de esta lógica es la *terminación*

“{P} S {Q}”

predicados lógicos que deben cumplir:

- **P** - precondición
- **S** - programa
- **Q** - postcondición

estado válido descrito por **P**, el programa lo hace en un estado válido que cumple la postcondición **Q**.

recondición(P) - postcondición(Q)

se por contrato.

donde P y Q son predicados lógicos que deben cumplirse

“{P} S {Q}”

goritmos

UMSS

Conocer conciencia de las condiciones del problema.

Almente son las condiciones para los textos previos $\{P\}$

Almente son las condiciones que las y/o contextos posteriores $\{S\}$

as con
 $\{P\}$
as con
xtos p

 $\{P\}$ $\{Q\}$

to:

to

$$\frac{\{P\} S \{Q\}; P' \Rightarrow P}{\vdash \{P'\} S \{Q\}}$$

$$\frac{\{P\} S \{Q\}; Q \Rightarrow Q'}{\vdash \{P\} S \{Q'\}}$$

- ## ► Debilitamiento

$$\frac{\{P\} S \{Q\}; P' \Rightarrow P}{\vdash \{P'\} S \{Q\}}$$

$$\frac{\{P\} \ S \ \{Q\}; \ Q \Rightarrow Q'}{\vdash \{P\} \ S \ \{Q'\}}$$

la asignación, x cumple la misma propiedad antes de ejecutarse la instrucción

$$\frac{P \Rightarrow Q_x^t}{\{P\} \ x:=t \ \{Q\}}$$

este caso se refuerza la propiedad

$$\frac{}{\vdash \{P_x^t\} \quad x:=t \quad \{P\}}$$
$$\frac{P \Rightarrow Q_x^t}{\{P\} \quad x := t \quad \{Q\}}$$

◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

UMSS

$$\frac{\vdash \{P \wedge B\} \alpha \{Q\}}{\vdash \{P\} \text{ while } B \text{ do } \alpha \text{ elihw } \{P \wedge \sim B\}}$$
$$\frac{\vdash \{P \wedge B\} \alpha \{Q\}}{\vdash \{P\} \text{ while } B \text{ do } \alpha \text{ elihw } \{P \wedge \sim B\}}$$

de generar ejecuciones infinitas,
modo que permita mostrar la evolución
de la condición de control, a través

le B do α elihw

- $I \wedge B \Rightarrow f \geq 0$
- $\{I \wedge B \wedge f = A\} \alpha \{f < A\}$

valor positivo

while B do α elihw

2. $\vdash \{I \wedge B \wedge f = A\} \alpha \{f < A\}$

◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ 🔍 ↺

Formula las pre y pos condiciones para los siguientes ejercicios

Algoritmos

Se desea encontrar la suma.

- $\{x, y \in \mathbb{Z}; x = A; y = B\}$
- $\{s \in \mathbb{Z}; s = A + B\}$

general:

- $\{x, y \in \text{Numero}; x = A; y = B\}$
- $\{s \in \text{Numero}; s = A + B\}$

Example (1)

Dados dos números se desea encontrar la suma.

$$P \equiv \{x, y \in \mathbb{Z}; x = A; y = B\}$$

$$Q \equiv \{s \in \mathbb{Z}; s = A + B\}$$

Otra opción mas general:

$$P \equiv \{x, y \in \text{Numero}; x = A; y = B\}$$

$$Q \equiv \{s \in \text{Numero}; s = A + B\}$$

Se desea encontrar la división

- $\{x, y \in \mathbb{Z}; x = A; y = B; y \neq 0\}$
- $\{c \in \mathbb{Z}; c = A/B\}$

general:

- $\{x, y \in \text{Numero}; x = A; y \neq 0\}$
- $\{c \in \text{Numero}; c = A/B\}$

Example (2)

Dados dos números se desea encontrar la división.

$$P \equiv \{x, y \in \mathbb{Z}; x = A; y = B; y \neq 0\}$$

$$Q \equiv \{c \in \mathbb{Z}; c = A/B\}$$

Otra opción mas general:

$$P \equiv \{x, y \in \text{Numero}; x = A; y = B; y \neq 0\}$$

$$Q \equiv \{c \in \text{Numero}; c = A/B\}$$

goritmos

os se desea intercambiar sus va

$\{x, y \in \mathbb{Z}; x = A; y = B\}$

$\{x = B; y = A\}$

Example (3)

Dados dos números se desea intercambiar sus valores.

$$P \equiv \{x, y \in \mathbb{Z}; x = A; y = B\}$$

$$Q \equiv \{x = B; y = A\}$$

Algoritmos

Para la raíz cuadrada entera de un número natural n , siempre es posible encontrar la raíz cuadrada entera de n .

$\mathbb{Z} = \{x \in \mathbb{Z}; x = A * A; x \geq 0\}$
 $\mathbb{Z} = \{r \in \mathbb{Z}; r = A\}$

Se desea encontrar la raíz cuadrada entera de un número entero positivo x .

$$P \equiv \{x \in \mathbb{Z}; x = A * A; x \geq 0\}$$

$$Q \equiv \{r \in \mathbb{Z}; r = A\}$$

Correctitud de Algoritmos

Example (5)

Dado dos números enteros ordénalos ascendentemente

$$P \equiv \{x, y \in \mathbb{Z}; x = A; y = B\}$$

$$Q \equiv \{x = \min(A, B); y = \max(A, B)\}$$

Correctitud de Algoritmos

Example (6)

Dado tres números enteros ordénelos ascendentemente

Correctitud de Algoritmos

Example (6)

Dado tres números enteros ordénelos ascendentemente

$$P \equiv \{x, y, z \in \mathbb{Z}; x = A; y = B; z = C\}$$

$$Q \equiv \{(a, b, c) = \text{perm}(A, B, C) \text{ tal que } : a \leq b \leq c\}$$

Verificar

Example (1)

Dados dos números se desea encontrar la suma.

$$P \equiv \{x, y \in \mathbb{Z}; x = A; y = B\}$$

```
1  sumar(x, y : entero)
2  inicio
3    s : entero
4    s = x+y
5    retornar s
6  fin
```

$$Q \equiv \{s \in \mathbb{Z}; s = A + B\}$$

Verificar

Example (4)

Se desea encontrar la raíz cuadrada entera de un número entero positivo x .

Considera que siempre es posible encontrar la respuesta.

$$P \equiv \{x \in \mathbb{Z}; x = A * A; x \geq 0\}$$

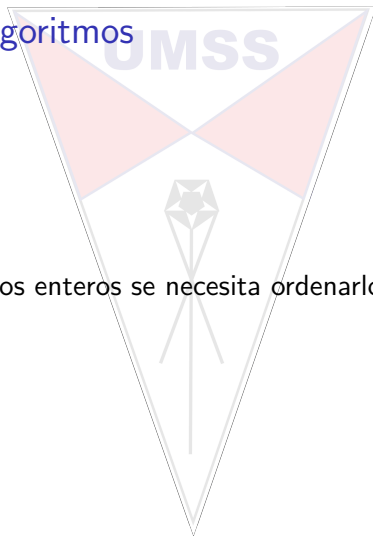
```
1  raizEntera(x: entero)
2  inicio
3    r : entero
4    r = sqrt(x)
5  retornar r
6  fin
```

$$Q \equiv \{r \in \mathbb{Z}; r = A\}$$

Correctitud de Algoritmos

Example (2)

Dados tres números enteros se necesita ordenarlos ascendentemente.



$P \equiv \{x, y, z \in \mathbb{Z}; x = A; y = B; z = C\}$

```
1  ordenar(x, y, z: entero)
2  inicio
3    a, b, c: entero
4    si(x < y)
5      entonces si(y < z)
6        entonces a = x, b = y, c = z
7      sino si(x < z)
8        entonces a = x, b = z, c = y
9      sino a = z, b = x, c = y
10   sino si(x < z)
11     entonces a = y, b = x, c = z
12   sino si(y < z)
13     entonces a = y, b = z, c = x
14   sino a = z, b = y, c = x
15   retornar a, b, c
16 fin
```

$Q \equiv \{(a, b, c) = \text{perm}(A, B, C) \text{ tal que } : a \leq b \leq c\}$

Correctitud de iteraciones *mientras*

Se pide realizar un proceso que permita multiplicar dos números enteros mediante sumas.

La especificación general será:

$x, y, prod : \text{entero}$
 $\{x = X; y = Y; y \geq 0\}$
S
 $\{prod = XY\}$

Correctitud de iteraciones *mientras*

El proceso:

$P \equiv \{x = X; y = Y; y \geq 0; x, y \in \mathbb{Z}\}$

```
1  multiplicar(x, y : entero)
2  inicio
3      prod : entero
4      prod = 0
5      mientras(y != 0) hacer
6          prod = prod + x
7          y = y - 1
8      fmientras
9      retornar prod
10 fin
```

$Q \equiv \{prod = XY; prod \in \mathbb{Z}\}$

Correctitud

¿ T ? . Se requiere definir tiempo estimado de realización de la repetición

$$P \equiv \{x = X; y = Y; y \geq 0; x, y \in \mathbb{Z}\}$$

```

1  multiplicar(x, y : entero)
2  inicio
3      prod : entero
4      prod = 0
5      mientras(y != 0) hacer ← AQUÍ
6          prod = prod + x
7          y = y - 1
8      fmientras
9      retornar prod
10 fin
  
```

$$Q \equiv \{prod = XY; prod \in \mathbb{Z}\}$$

T ≡ Y; t ≡ T; ira disminuyendo hasta llegar a 0

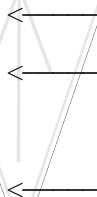
Correctitud

¿ INVARIANTE $\equiv I$?. Se requiere definir predicados que permitan ser válidos: antes, mientras y al final de la iteración

$$P \equiv \{x = X; y = Y; y \geq 0; x, y \in \mathbb{Z}\}$$

```

1  multiplicar(x, y : entero)
2  inicio
3      prod : entero
4      prod = 0
5      I
6      mientras(y != 0) hacer
7          I
8          prod = prod + x
9          y = y - 1
10     fmientras
11     I
12     retornar prod
13 fin
  
```



$$Q = \{prod = XY, prod \in \mathbb{Z}\}$$

Correctitud

$$P \equiv \{x = X; y = Y; y \geq 0; x, y \in \mathbb{Z}\}$$

```

1  multiplicar(x, y : entero)
2  inicio
3      prod : entero
4      prod = 0
5      l: {XY = prod + xy; y >= 0}
6      mientras(y != 0) hacer
7          l: {XY = prod + xy; y >= 0}
8          prod = prod + x
9          y = y - 1
10     fmientras
11     l: {XY = prod + xy; y >= 0}
12     retornar prod
13 fin

```

$$Q \equiv \{prod = XY; prod \in \mathbb{Z}\}$$

Verificar cumplimiento, verificar fin de ejecución...

Práctica

Determina el invariante y la función del tiempo de:

$P \equiv \{x = X; y = Y; y \geq 0; y = 0 \Rightarrow x \neq 0; x, y : \text{entero}\}$

```
1 potencia(x, y : entero)
2 inicio
3   z : entero
4   z = 1
5   mientras(y != 0) hacer
6     z = z * x
7     y = y - 1
8   fmientras
9   retornar z
10 fin
```

$Q \equiv \{z = X^Y; z : \text{entero}\}$

Práctica

Determina el invariante y la función del tiempo de:

$P \equiv \{x = X; y = Y; y \geq 0; y = 0 \Rightarrow x \neq 0; x, y : \text{entero}\}$

```

1 potencia(x, y : entero)
2 inicio
3   z : entero
4   z = 1
5   mientras(y != 0) hacer
6     z = z * x
7     y = y - 1
8   fmientras
9   retornar z
10 fin
  
```

$Q \equiv \{z = X^Y; z : \text{entero}\}$

El invariante:

$\{X^Y = z * x^y \wedge y \geq 0\}$

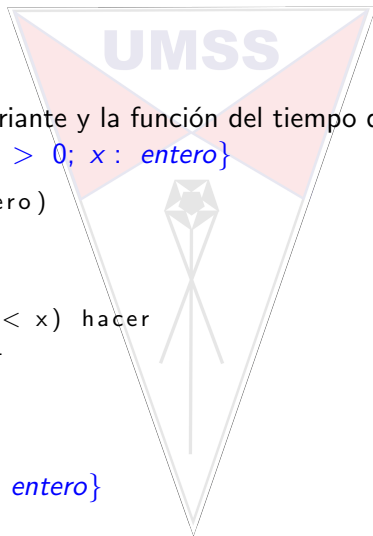
Práctica

Determina el invariante y la función del tiempo de:

$$P \equiv \{x = X; x > 0; x : \text{entero}\}$$

```
1  sumar(x: entero)
2  inicio
3    n : entero
4    n = 1
5    mientras(n < x) hacer
6      n = n + 1
7    fmientras
8    retornar n
9  fin
```

$$Q \equiv \{n = X; n : \text{entero}\}$$

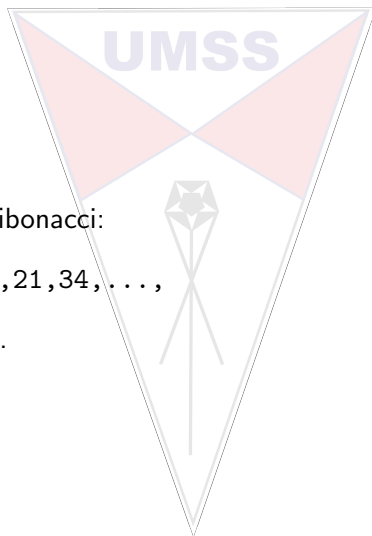


Ejemplo

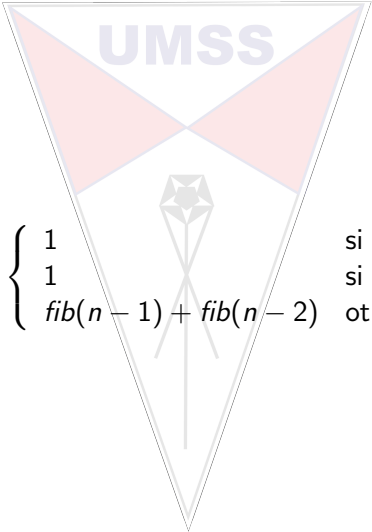
Dada la serie de fibonacci:

1, 1, 2, 3, 5, 8, 13, 21, 34, ...,

y su definición



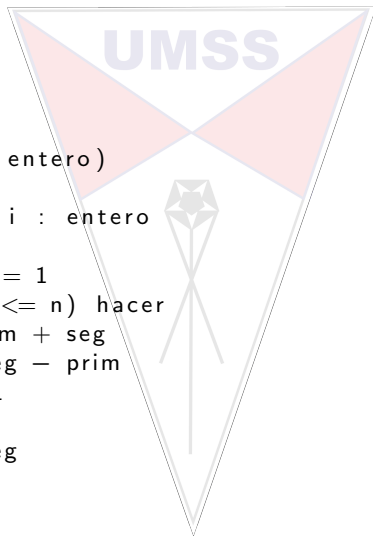
Ejemplo


$$fib(n) = \begin{cases} 1 & \text{si } n = 1 \\ 1 & \text{si } n = 2 \\ fib(n-1) + fib(n-2) & \text{otro caso} \end{cases}$$

Correctitud

O esta opción:

```
1  fibonacci(n: entero)
2  inicio
3      prim, seg, i : entero
4      i = 3
5      prim = seg = 1
6      mientras(i <= n) hacer
7          seg = prim + seg
8          prim = seg - prim
9          i = i + 1
10     fmientras
11     retornar seg
12 fin
```



- ▶ Es correcto?
- ▶ Cómo formalizas?

20' para hacerlo....

1. P
2. $\{P\}$ Instruccion $\{Q\}$

1. P
2. $\{P \wedge B\} \text{ CuerpoSI } \{Q\}$
3. $\{P \wedge \neg B\} \text{ CuerpoNO } \{Q\}$

$\}$
 $\{Q\}$
 $t > 0\}$
 $T\}S\{t < T\}$

1. $P \Rightarrow I$
2. $\{I \wedge B\} S \{I\}$
3. $\{I \wedge \sim B\} \Rightarrow \{Q\}$
4. $\{I \wedge B\} \Rightarrow t > 0\}$
5. $\{I \wedge B \wedge t = T\} S \{t < T\}$

UMSS

ORMEN CHARLES E. LEISERSON
IN (2009), *INTRODUCTION TO A*
MIT Press

